

Build a CNN to Classify CIFAR-10 Images (PyTorch)

Build a CNN to Classify CIFAR-10 Images (PyTorch)

Objective

Create a **CNN (Convolutional Neural Network)** that classifies images from the **CIFAR-10 dataset** into 10 categories and achieves more than **60% accuracy**.

Import Libraries

Import Libraries

Code	Purpose
<code>import torch</code>	Main PyTorch library
<code>import torch.nn as nn</code>	Neural network layers
<code>import torch.optim as optim</code>	Optimization algorithms
<code>import torchvision</code>	Image datasets and utilities

<code>import torchvision.transforms as transforms</code>	Image preprocessing functions
--	-------------------------------------

Image Preprocessing

```
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize((0.5,), (0.5,))  
])
```

Explanation

Code	Purpose
<code>Compose()</code>	Combines multiple preprocessing steps
<code>ToTensor()</code>	Converts image to tensor
<code>Normalize()</code>	Scales pixel values for faster learning

Data Flow

```
Image  
↓  
ToTensor()  
↓  
Normalize()
```

↓
Ready for CNN

Load CIFAR-10 Dataset

```
trainset = torchvision.datasets.CIFAR10(...)  
testset = torchvision.datasets.CIFAR10(...)
```

CIFAR-10 Details

Property	Value
Training Images	50,000
Test Images	10,000
Classes	10
Image Size	32×32
Channels	RGB (3 Channels)

Classes

Label	Class
0	Airplane

- 1 Autom
 obile
 - 2 Bird
 - 3 Cat
 - 4 Deer
 - 5 Dog
 - 6 Frog
 - 7 Horse
 - 8 Ship
 - 9 Truck
-

DataLoader

trainloader = DataLoader(...)

testloader = DataLoader(...)

Parameter	Meaning
------------------	----------------

<code>batch_size=64</code>	Process 64 images at a time
----------------------------	-----------------------------

<code>shuffle=True</code>	Randomize training images
---------------------------	---------------------------

`shuffle` Keep test images
`=False` in order

CNN Architecture

```
class Net(nn.Module):
```

Creates a CNN model.

Convolution Block

```
self.conv = nn.Sequential(  
    nn.Conv2d(3, 32, 3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2),  
  
    nn.Conv2d(32, 64, 3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2)  
)
```

Layer-by-Layer

Layer	Input	Output	Purpose
-------	-------	--------	---------

Conv 2D	3 Chann els	32 Filters	Extract features
ReLU	32	32	Add non-linearity
MaxP ool	32×32	16×1 6	Reduce image size
Conv 2D	32 Filters	64 Filters	Learn deeper features
ReLU	64	64	Activation
MaxP ool	16×16	8×8	Reduce size again

Image Size Transformation

32×32×3
 ↓ Conv
 32×32×32
 ↓ Pool
 16×16×32
 ↓ Conv
 16×16×64
 ↓ Pool
 8×8×64

Fully Connected Layers

```
self.fc = nn.Sequential(  
    nn.Linear(64*8*8, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10)  
)
```

Explanation

Lay er	Inp ut	Outpu t
-----------	-----------	------------

Line ar	4096	128
------------	------	-----

ReL U	128	128
----------	-----	-----

Line ar	128	10 Class es
------------	-----	-------------------

Why 4096?

$$64 \times 8 \times 8 = 4096$$

After convolution and pooling:

64 feature maps

8 × 8 size

Forward Pass

```
def forward(self, x):
```

Step-by-Step

Code	Purpose
<code>x = self.conv(x)</code>	Extract features
<code>x.view(x.size(0), -1)</code>	Flatten data
<code>self.fc(x)</code>	Predict class

Create Model

```
net = Net()
```

Creates CNN object.

Loss Function

```
criterion = nn.CrossEntropyLoss()
```

Purpose

Measures prediction error.

Example:

Actual	Predicted
--------	-----------

Dog	Cat
-----	-----

Loss becomes high.

Optimizer

```
optimizer = optim.Adam(net.parameters(), lr=0.001)
```

Purpose

Updates weights to reduce loss.

Parameter	Meaning
-----------	---------

Adam	Popular optimizer
------	-------------------

0.001	Learning rate
-------	---------------

Training Loop

```
for epoch in range(1):
```

Train for 1 epoch.

Forward Pass

`outputs = net(images)`

Prediction.

Calculate Loss

`loss = criterion(outputs, labels)`

Compares prediction with actual labels.

Clear Old Gradients

`optimizer.zero_grad()`

Removes previous gradients.

Backpropagation

`loss.backward()`

Calculates gradients.

Update Weights

`optimizer.step()`

Improves model weights.

Complete Training Flow

Images



CNN



Predictions



Loss Calculation



Backpropagation



Weight Update



Repeat

Interview Questions

Question	Answer
What is CNN?	A deep learning model specialized for image processing.
Why use Conv2D?	To automatically extract image features.
Why use ReLU?	To introduce non-linearity.

Why use
MaxPooling?

To reduce image size and
computation.

Why use
CrossEntropyLoss?

Suitable for multi-class
classification.

Why use Adam
optimizer?

Faster and efficient optimization.

What does
`loss.backward()`
do?

Computes gradients for all trainable
parameters.

What does
`optimizer.step()`
do?

Updates weights using calculated
gradients.

One-Line Summary

This code loads the CIFAR-10 dataset, preprocesses images, builds a CNN with convolution and fully connected layers, and trains the model using Adam optimizer and CrossEntropy loss to classify images into 10 categories.

=====

CIFAR-10 CNN Code Explanation (Table Format)

Section	Code	Purpose	Explanation
Import	<code>import torch</code>	PyTorch Library	Main deep learning framework.
Import	<code>import torch.nn as nn</code>	Neural Network Module	Provides layers like Conv2D, Linear, ReLU.
Import	<code>import torch.optim as optim</code>	Optimizer Module	Used to update model weights.
Import	<code>import torchvision</code>	Dataset Utilities	Provides CIFAR-10 dataset.
Import	<code>import torchvision.transforms as transforms</code>	Preprocessing	Used for image transformations.

Preprocessing	<code>transforms.Compose()</code>	Combine Steps	Groups multiple preprocessing operations.
Preprocessing	<code>transforms.ToTensor()</code>	Convert Image	Converts image to PyTorch tensor.
Preprocessing	<code>transforms.Normalize((0.5,), (0.5,))</code>	Normalize Data	Scales pixel values for better training.
Dataset	<code>CIFAR10(train=True)</code>	Training Data	Loads 50,000 training images.
Dataset	<code>CIFAR10(train=False)</code>	Test Data	Loads 10,000 testing images.
DataLoader	<code>batch_size=64</code>	Batch Processing	Processes 64 images at a time.
DataLoader	<code>shuffle=True</code>	Random Training	Randomizes image order

			during training.
CNN Layer	<code>nn.Conv2d(3, 32, 3, padding=1)</code>	Feature Extraction	Extracts basic image features.
Activation	<code>nn.ReLU()</code>	Non-Linearity	Helps model learn complex patterns.
Pooling	<code>nn.MaxPool2d(2)</code>	Reduce Size	Reduces image dimensions by half.
CNN Layer	<code>nn.Conv2d(32, 64, 3, padding=1)</code>	Deep Features	Learns more advanced features.
Activation	<code>nn.ReLU()</code>	Non-Linearity	Improves learning capability.
Pooling	<code>nn.MaxPool2d(2)</code>	Reduce Size	Further reduces image size.
Fully Connected	<code>nn.Linear(64*8*8, 128)</code>	Hidden Layer	Converts extracted features into

			meaningful patterns.
Activation	<code>nn.ReLU()</code>	Activation	Introduces non-linearity.
Output Layer	<code>nn.Linear(128, 10)</code>	Classification	Predicts one of 10 CIFAR-10 classes.
Forward Pass	<code>x=self.conv(x)</code>	Feature Extraction	Sends image through convolution layers.
Flatten	<code>x.view(x.size(0), -1)</code>	Reshape Data	Converts 3D feature maps into 1D vector.
Prediction	<code>return self.fc(x)</code>	Final Output	Produces class scores.
Model Creation	<code>net=Net()</code>	Create CNN	Instantiates CNN model.
Loss Function	<code>nn.CrossEntropyLoss()</code>	Calculate Error	Measures prediction accuracy.

Optimizer	<code>optim.Adam(...,lr=0.001)</code>	Update Weights	Optimizes model parameters.
Training	<code>for epoch in range(1)</code>	Training Loop	Runs one complete training cycle.
Prediction	<code>outputs=net(images)</code>	Forward Pass	Generates predictions.
Loss	<code>loss=criterion(outputs,labels)</code>	Error Calculation	Compares prediction with actual label.
Gradient Reset	<code>optimizer.zero_grad()</code>	Clear Gradients	Removes previous gradients.
Backpropagation	<code>loss.backward()</code>	Compute Gradients	Calculates weight updates.
Weight Update	<code>optimizer.step()</code>	Learn	Updates model weights.

=====

CNN Architecture Summary

Layer	Input Shape	Output Shape
Input Image	$3 \times 32 \times 32$	$3 \times 32 \times 32$
Conv2D (3→32)	$3 \times 32 \times 32$	$32 \times 32 \times 32$
MaxPool	$32 \times 32 \times 32$	$32 \times 16 \times 16$
Conv2D (32→64)	$32 \times 16 \times 16$	$64 \times 16 \times 16$
MaxPool	$64 \times 16 \times 16$	$64 \times 8 \times 8$
Flatten	$64 \times 8 \times 8$	4096
Linear	4096	128
Linear	128	10 Classes

=====

CIFAR-10 Dataset Summary

Property	Value
Training Images	50,000
Testing Images	10,000
Number of Classes	10
Image Size	32 × 32 Pixels
Channels	RGB (3 Channels)
Batch Size Used	64
Optimizer	Adam
Learning Rate	0.001
Loss Function	CrossEntropyLoss

=====

Question

Answer

What is CNN?	A deep learning model used for image classification and feature extraction.
Why Conv2D?	To automatically detect image features such as edges, shapes, and objects.
Why MaxPooling?	To reduce image size and computation.
Why ReLU?	To introduce non-linearity and improve learning.
Why CrossEntropyLoss?	Suitable for multi-class classification problems.
Why Adam Optimizer?	Provides faster and efficient weight updates.